

Day 28: Weekly Revision

1. Day Number

Day 28

2. Topic Name

Revision of Days 22–27

3. Connection

During Days 22–27, you learned several small programming patterns:

- Limiting login attempts
- Searching for a payee
- Calculating totals
- Storing fixed transaction records
- Printing dictionary data
- Handling invalid input safely

Today, you will combine **safe input, tuples, functions, conditions, and formatted output** in one small program.

4. Revision Summary of Days 22–27

Day 22: Limited Login Attempts

You used:

- A while loop
- A counter
- A condition
- break

Basic pattern:

```
attempts = 0

while attempts < 3:
    # Ask for password
    # Check password
    attempts += 1
```

The counter prevents the loop from running forever.

Day 23: Search Payee by Name

You searched through a list of dictionaries.

Basic pattern:

```
found = False

for payee in payees:
    if payee["name"] == search_name:
        found = True
        break
```

The Boolean flag remembers whether a matching record was found.

Day 24: Calculate Total Deposits

You used an accumulator variable.

Basic pattern:

```
total = 0

for amount in deposits:
    total = total + amount
```

The accumulator starts at 0 and keeps collecting values.

Day 25: Transaction Tuple Record

You stored one fixed transaction as a tuple.

```
transaction = ("Deposit", 500)
```

Individual values can be accessed using indexes:

```
transaction[0]
transaction[1]
```

A tuple is useful when the structure of a record should remain fixed.

Day 26: Print Dictionary Using .items()

You printed dictionary keys and values using a loop.

```
for key, value in profile.items():
    print(key, value)
```

The .items() method gives both the key and its corresponding value.

Day 27: Safe Numeric Input

You used try-except to prevent the program from crashing when the user enters invalid data.

```
try:
    amount = float(input("Enter amount: "))
except ValueError:
    print("Please enter a valid number.")
```

`ValueError` occurs when Python cannot convert the entered text into a number.

5. Important Topics

while Loop

Repeats code while a condition remains true.

```
while condition:
    # repeated work
```

Counter

Tracks how many times something has happened.

```
attempts = attempts + 1
```

Search Pattern

Checks records one by one until a match is found.

```
for item in records:
    if item matches:
        # process it
```

Accumulator

Stores a running total.

```
total = total + amount
```

Tuple

Stores an ordered, fixed group of values.

```
transaction = ("Deposit", 250)
```

.items()

Returns dictionary keys and values together.

```
for key, value in data.items():
    print(key, value)
```

try-except

Handles errors without stopping the complete program.

```
try:
    # risky conversion
except ValueError:
    # handle invalid input
```

6. Foundational Notes

Input is always received as text

Even when a user enters 500, Python initially receives it as a string:

```
amount_text = input("Enter amount: ")
```

To perform calculations, convert it:

```
amount = float(amount_text)
```

Conversion can fail

The user might enter:

```
five hundred
```

Python cannot convert this text using `float()`, so it raises a `ValueError`.

That is why conversion should be placed inside `try-except`.

Validate after successful conversion

`try-except` checks whether the input is numeric. A separate condition checks whether the number is acceptable.

```
if amount > 0:
    # valid deposit
else:
    # zero or negative deposit
```

These are different checks:

- `try-except`: Is the input a number?
 - `if`: Is the number a valid deposit?
-

A transaction tuple can contain related values

For example:

```
("Deposit", 500)
```

Here:

- Index 0 contains the transaction type.
 - Index 1 contains the amount.
-

Functions organize responsibilities

A function should perform one clear task.

Possible functions:

```
def get_deposit_amount():  
    pass
```

```
def create_transaction(amount):  
    pass
```

```
def print_transaction(transaction):  
    pass
```

You do not need all three functions, but separating input and display logic makes the program easier to understand.

7. Easy Example

This example demonstrates safe input only. It is not the complete revision solution.

```
try:  
    age = int(input("Enter your age: "))  
  
    if age > 0:  
        print("Valid age")  
    else:  
        print("Age must be positive")  
  
except ValueError:  
    print("Please enter age using numbers.")
```

Example run with valid input

```
Enter your age: 25  
Valid age
```

Example run with invalid input

```
Enter your age: twenty  
Please enter age using numbers.
```

What happens here?

1. The program asks for input.
 2. It tries to convert the input into an integer.
 3. If conversion succeeds, it validates the number.
 4. If conversion fails, the except block runs.
-

8. Revision Problem Statement

Create a small Python program that:

1. Safely asks the user to enter one deposit amount.
2. Uses try-except to handle invalid numeric input.

3. Accepts the deposit only when the amount is greater than 0.
4. Stores a successful transaction as a tuple containing:
 - Transaction type
 - Deposit amount
5. Prints the transaction type and amount separately.
6. Prints a suitable message for invalid, zero, or negative amounts.

Example tuple structure:

```
("Deposit", amount)
```

Do not write the complete solution immediately. First divide the program into smaller tasks.

9. Concepts Used

- `input()`
 - `float()`
 - Functions
 - Function parameters
 - Return values
 - `try-except`
 - `ValueError`
 - `if-else`
 - Comparison operators
 - Tuple creation
 - Tuple indexing
 - Formatted output
-

10. Thought Process

Before writing code, think through the program step by step.

Step 1: Identify the risky operation

Converting user input to `float` can fail.

Therefore, conversion belongs inside `try`.

Step 2: Decide what happens for invalid text

If the user enters something like `abc`, the `except ValueError` block should print an error message.

Step 3: Validate the converted amount

After conversion succeeds, check:

```
Is the amount greater than zero?
```

- Greater than 0: Create the transaction.
- Equal to 0: Reject it.
- Less than 0: Reject it.

Step 4: Create the tuple

A valid deposit can be stored as:

```
("Deposit", entered_amount)
```

Step 5: Print tuple values

Use tuple indexes to print the values separately:

```
Transaction type: Deposit  
Amount: $500.00
```

Step 6: Organize the program using functions

A beginner-friendly functional structure could be:

```
Function 1: Safely read and validate the deposit  
Function 2: Print the transaction details  
Main section: Call the functions
```

11. Pseudocode

```
DEFINE function get_deposit_amount

    TRY
        ASK user for deposit amount
        CONVERT input to decimal number

        IF amount is greater than zero
            RETURN amount
        ELSE IF amount equals zero
            PRINT deposit must be greater than zero
        ELSE
            PRINT deposit cannot be negative

    EXCEPT invalid number error
        PRINT please enter a valid number

    RETURN no valid value

<div class="source-blank-line"></div>

DEFINE function print_transaction that receives amount

    CREATE transaction tuple with:
        transaction type "Deposit"
        amount

    PRINT transaction type using tuple index
    PRINT transaction amount using tuple index

<div class="source-blank-line"></div>

CALL get_deposit_amount
STORE returned result

IF returned result is valid
    CALL print_transaction with result
```

12. Suggested Solving Approach: Functional Approach

Function 1: Get the deposit amount

Suggested responsibility:

- Ask for input
- Convert the input
- Handle ValueError
- Check whether the amount is positive
- Return the valid amount

Possible function outline:

```
def get_deposit_amount():  
    # try-except  
    # validation  
    # return valid amount  
    pass
```

Function 2: Display the transaction

Suggested responsibility:

- Receive a valid amount
- Create a tuple
- Print tuple values separately

Possible function outline:

```
def display_transaction(amount):  
    # create tuple  
    # print transaction[0]  
    # print transaction[1]  
    pass
```

Main program

The main section should:

1. Call the input function.
2. Store its returned result.
3. Check whether a valid amount was returned.
4. Call the display function only for a valid amount.

This prevents an invalid transaction from being created.

13. Easy Edge Cases

Edge Case 1: Invalid Amount

Input:

```
Enter deposit amount: hello
```

Expected result:

```
Please enter a valid numeric amount.
```

The program should not crash.

Edge Case 2: Amount 0

Input:

```
Enter deposit amount: 0
```

Expected result:

```
Deposit amount must be greater than zero.
```

No transaction tuple should be created.

Edge Case 3: Negative Amount

Input:

```
Enter deposit amount: -500
```

Expected result:

```
Deposit amount cannot be negative.
```

No transaction tuple should be created.

Edge Case 4: Decimal Amount

Input:

```
Enter deposit amount: 250.75
```

Expected result:

```
Transaction type: Deposit  
Amount: $250.75
```

Using `float()` allows decimal values.

14. Common Mistakes to Avoid

Mistake 1: Converting outside the `try` block

Incorrect pattern:

```
amount = float(input("Enter amount: "))  
  
try:  
    # other code
```

The conversion error occurs before Python reaches `try`.

Place the conversion inside `try`.

Mistake 2: Catching the wrong error

For invalid numeric conversion, use:

```
except ValueError:
```

Avoid using an unrelated exception type.

Mistake 3: Creating a transaction before validation

Do not create a transaction for 0 or a negative amount.

Validate first, then create the tuple.

Mistake 4: Using the wrong tuple index

For:

```
transaction = ("Deposit", 500)
```

- `transaction[0]` is "Deposit".
- `transaction[1]` is 500.

`transaction[2]` would cause an `IndexError` because there are only two values.

Mistake 5: Forgetting to return from the function

A function may calculate or validate the amount correctly but still return `None` if `return` is missing.

Mistake 6: Comparing a string with a number

This is incorrect:

```
amount = input("Enter amount: ")  
  
if amount > 0:
```

Convert the input before performing numeric comparisons.

Mistake 7: Printing transaction details for invalid input

Only call the transaction display function when a valid positive amount is available.

15. Quick Self-Check Questions

1. Why should `float(input(...))` be placed inside a `try` block?
 2. What exception occurs when the user enters "abc" and Python tries to convert it using `float()`?
 3. In the tuple `("Deposit", 500)`, what does index 1 contain?
 4. What is the difference between invalid text input and a negative numeric input?
 5. Why should the transaction tuple be created only after validation?
-

16. Hint Only

Start with a function that safely returns either:

- A positive deposit amount, or

- A special value such as None when the input is invalid

Then check the returned value before creating this tuple:

```
("Deposit", amount)
```

Use indexes 0 and 1 to print the two transaction details separately.